Automated Visual Inspection of Packaged Pork Rashers Using YOLOv8 Object Detection

MMA3001 - *Numerical Methods and Machine Learning*
Individual Project Documentation
Bradley Medlock - 34946888

1. Engineering Problem and Project Objectives

1.1 Engineering Context

Quality control is an important part of food manufacturing, where products must be inspected for visual defects prior to distribution. Traditionally, this visual inspection has relied on human observation, creating an opportunity for computer vision to assist in identifying inconsistencies. Machine learning based object detection offers a method of analysing product images to identify and locate visible defects, making it a relevant computational approach for automated packaging inspection.

1.2 Problem Definition and Significance

The engineering problem is to develop a computer vision based inspection system for detecting and locating visual defects in packaged pork rashers. Inconsistent identification of these defects may impact product presentation and packaging quality. Automated detection could improve inspection consistency and assist in identifying defective products for further review.

1.3 Project Objectives

The primary objective of this project is to develop and evaluate a YOLOv8 based object detection model for identifying visual defects in packaged pork rashers. The specific objectives are to

- Train and evaluate multiple YOLOv8 model configurations.
- Compare detection performance across different model sizes and image resolutions.
- Assess computational requirements, including inference time.
- Identify the most suitable configuration based on performance and computational cost.
- Evaluate the model's limitations and suitability for automated visual inspection.

1.4 Intended Application and Scope Limits

The intended application is an automated visual inspection that analyses images of packaged pork rashers and identifies defects for further review. The investigation is limited to the supplied image dataset and offline model evaluation. Physical integration with a production line, real time conveyor inspection or food safety inspection is not within the scope of this project. The results demonstrate the model’s performance on available images rather than its readiness for industrial deployment and application.

2. Inputs, Outputs and Dataset

2.1 Dataset Source and Description

The dataset used in this project, “Pork Rasher Error(Packaging) Computer Vision Dataset” (Version 4), was obtained through Roboflow Universe [4]. It contains 3549 images, divided into training (3349 images), validation (120 images) and test (80 images) subsets. The data was exported in YOLOv8 format, providing images and corresponding annotation files for object detection model training and evaluation.

2.2 Defect Classes and Annotation Structure

The dataset contains five defect classes: loose-meat, packaging-error, twisted-meat, unsealed and wrinkle. Each image (with defects) is annotated with bounding boxes to indicate the location of labelled defects. Annotations are stored in YOLO format, where each defect bounding box is represented by a class identifier, center coordinates, width and height [1]. This allows the constructed model to learn defect classification and localisation.

2.3 Dataset Distribution and Quality

The dataset exhibits a substantial imbalance in defect class distribution. An audit of the dataset identified 3,123 annotated bounding boxes across 3,549 images. The distribution is highly imbalanced, with packaging-error defects accounting for 56.64% of annotations, and unsealed accounting for 37.94%. The remaining classes are substantially less represented, particularly wrinkle, which accounts for only 0.58% of annotations.

The audit also identified 710 empty annotation files, which were retained as negative examples. No missing annotation files or invalid annotations were identified. Notably, the validation and test splits contain no annotated wrinkle instances, preventing meaningful evaluation of the models ability to identify this defect.

2.4 Computational Inputs and Output

The model takes RGB images of packaged pork rashers as an input, together with their corresponding annotations during training and evaluation. Images are processed at specific input resolutions, while annotations provide the defect class and defect location (as bounding box coordinates).

The model makes predictions consisting of defect classifications, bounding box locations and associated confidence scores. These outputs identify the predicted defect type, its estimated position within the image and the model's confidence in that prediction.

3. Computational Solution

3.1 Object-Detection Methodology

The project uses object detection to identify and localise visible defects in packaged pork rasher images. During training, the model learns to associate visible features in the images with labelled defect classes and their corresponding bounding boxes. Once trained, it processes previously unseen images to predict the class and location of potential defects.

The approach uses supervised learning, as the training dataset provides labelled examples of each defect class and its corresponding location. Pretrained YOLOv8 weights are used as the starting point for training, allowing the model to adapt previously learned visual features to the specific defect categories in the dataset.

3.2 YOLOv8 Architecture and Model Configurations

YOLOv8, developed by Ultralytics, was selected as the object detection framework for this project. The architecture consists of three main components: a backbone, a neck and a detection head. The backbone extracts visual features from input images, while the neck combines features at different scales. The detection head uses these features to predict bounding box locations and corresponding defect classes [2].

Two model variants were investigated: YOLOv8n (Nano) and YOLOv8s (Small). These variants differ in model size and computational complexity, allowing the influence of model capacity on detection performance to be investigated.

Four configurations were compared, using different combinations of model variant and input resolution.

- YOLOv8n at 640 x 640 pixels.
- YOLOv8s at 640 x 640 pixels.
- YOLOv8s at 960 x 960 pixels.
- YOLOv8s at 1280 x 1280 pixels.

These specific model variations were selected to investigate the effects of model size and input resolution on detection accuracy and computational requirements. In particular, increasing image resolution was investigated to determine whether increasing image detail improves small defect detection. The resulting configurations were compared using detection performance metrics and inference time.

3.3 Training Workflow and Implementation

The models were implemented using Python and the Ultralytics YOLOv8 framework. Training was performed with pretrained YOLOv8 weights, subsequently fine tuned on the packaged pork rasher dataset. The dataset was provided in YOLO format, with separate training, validation and test splits.

Training was performed using Kaggle's GPU computing environment. Each configuration was trained using the same dataset and random seed to maintain cross model experimental consistency. Training was limited to 50 epochs, with early stopping patience of 10 epochs enabled to prevent unnecessary training once performance plateaued.

The best performing checkpoint was retained for each training run for subsequent comparison and evaluation. Model performance was assessed using the validation split, while the test split was reserved for evaluation of the final selected model.

3.4 Training Parameters and Assumptions

Table 3.1: Summary of Model Training Parameters and experimental configurations

Training Settings		**Hardware and Input Configurations**	
Parameter	**Setting**	**Parameter**	**Setting**
Maximum Epochs	50	GPU	Tesla T4
Patience	10	Device	0
Random Seed	42	Resolution	640, 960, 1280
Workers	2	Batch Size	16, 8, 4
Automatic Mixed Precision (AMP)		Enabled	

The experiment assumed the dataset annotations accurately represented the visible defects. The test subset was kept separate from the model selection to provide a final assessment of the selected configuration.

3.5 Evaluation Metrics

Model performance was evaluated using precision, recall, mean average precision at an intersection over union (IoU) at 50% (mAP50), and mean average precision averaged across IoU thresholds from 0.5 to 0.95 (mAP50-95). Precision measures the proportion of predicted detections that correctly identify labeled defects, while recall measures the proportion of actual defects successfully detected. Average precision summarizes the precision-recall relationship for an individual defect class, with mean average precision (mAP) calculated across all classes. The mAP50-95 provides a more comprehensive evaluation by measuring detection accuracy at increasingly strict levels of bounding box overlap [3]. Computational requirements were assessed using inference time, measured as the average time to process an individual image.

4. Alternative Solutions and Experimental Design

4.1 Baseline Model: YOLOv8n at 640px

The YOLOv8n (Nano) model trained at 640 x 640 pixels was established as the baseline configuration for the investigation. The nano variant was selected as a relatively lightweight model, providing a reference point for evaluating the effects of increased model capacity and image resolution on detection performance and computational requirements.

4.2 Increased Model Capacity: YOLOv8s at 640px

The YOLOv8s (Small) model trained at 640 x 640 pixels was introduced to investigate the effects of increased model capacity while maintaining consistent image resolution. The comparison aimed to determine whether the increased computational complexity of the small variant provided meaningful improvement in defect detection.

4.3 Increased Resolution: YOLOv8s at 960px

The YOLOv8s model trained at 960 x 960 pixels (batch size 8) introduced to investigate the effect of image resolution on detection performance. Model architecture was kept consistent with the YOLOv8s configuration, allowing the effect of increased image resolution to be examined in isolation. The experiment aimed to determine whether retaining additional image detail could improve the identification and localization of smaller defects, a weak point of prior models.

4.4 Further Resolution Increase: YOLOv8s at 1280px

Similarly to the previous experiment, the YOLOv8s model trained at an input image resolution of 1280 x 1280 pixels aimed to investigate if further increases in detail could provide further improvements in defect detection. The model architecture remained unchanged from the previous models for cross experiment consistency. The batch size was reduced to 4 to accommodate the increased GPU memory requirements.

4.5 Experimental Comparison and Model Selection

The four models were compared using validation performance, with particular consideration given to mAP50, mAP50-95 and inference time. This allowed detection accuracy to be assessed alongside computational costs associated with each configuration.

The YOLOv8s model at 640 x 640 pixel achieved the highest overall validation performance, with an mAP50 of 0.681 and an mAP50-95 of 0.424. The 960 x 960 pixel resolution model achieved higher recall, however its mean average precision and inference time was considerably worse than the lower resolution YOLOv8s model. Increasing resolution to 1280 x 1280 pixels did not improve overall detection performance.

From these results, the YOLOv8s model at 640 x 640 pixels was selected as the final configuration for evaluation on held out test split data. This selection provided the highest overall validation performance, while maintaining comparatively low computational requirements.

5. Validation and Results

5.1 Validation Strategy and Dataset Splitting

The four experimental configurations were evaluated using the same validation subset and consistent evaluation settings, with a confidence threshold of 0.001 and an IoU threshold of 0.6. This enabled their detection performance to be compared under equivalent conditions. The validation results were used to identify the most suitable configuration before conducting the final evaluation on the held out test set.

5.2 Validation Results

The validation results for the four experimental configurations are summarized in Table 5.1.

Table 5.1: Validation performance of the experimental configurations

Model	Resolution (px)	Precision	Recall	mAP50	mAP50-95	Inference (ms/image)
YOLOv8n	640	0.650	0.617	0.623	0.348	4.4 ms
YOLOv8s	640	0.781	0.619	0.681	0.424	7.9 ms
YOLOv8s	960	0.700	0.681	0.650	0.329	19.6 ms
YOLOv8s	1280	0.568	0.557	0.546	0.294	44.8 ms

5.3 Final Held-out Test-set Evaluation

Following model selection, the YOLOv8s configuration at 640 x 640 pixels was evaluated using the held out test subset, achieving a precision of 0.851, recall of 0.437, mAP50 of 0.570, mAP50-95 of 0.246, and an average inference time of 8.3 ms per image.

5.4 Per-Class Performance

Per class performance on the held out test dataset is summarized in Table 5.2.

Table 5.2: Per class detection performance on the test dataset

Defect class	Precision	Recall	mAP50	mAP50-95
Loose-meat	0.774	0.333	0.720	0.244
Packaging-error	0.773	0.565	0.572	0.132
Twisted-meat	1.000	0.000	0.122	0.037
Unsealed	0.856	0.851	0.865	0.573
Wrinkle	N/A	N/A	N/A	N/A

The Model performed best on unsealed defects, while twisted-meat detection was particularly poor. The wrinkle class could not be evaluated because no examples were present in the test subset.

5.5 Visual Error Analysis

A visual inspection of six test images was conducted to examine the model's predictions against original dataset annotations. The model correctly identified unsealed defects in all four instances. Of the two packaging-error examples, one was missed entirely while the other was detected with imperfect bounding-box location. The examples show that successful defect classification does not guarantee accurate localisation. The small sample examined is not representative of overall model performance.

6. Optimisation and Engineering Performance

6.1 Detection Accuracy and Computational Cost

Increasing model capacity from YOLOv8n to YOLOv8s at 640 x 640 pixels improved mAP50 from 0.623 to 0.681 and mAP-95 from 0.348 to 0.424, with inference time increasing from 4.4 to 7.9 ms/image. This represents an improvement in detection accuracy at the expense of increased computational cost. Despite the slower inference, the improved accuracy provided by YOLOv8s made it the stronger overall configuration for this project.

6.2 Effect of Model Complexity and Input Resolution

Increasing the input resolution from 640 x 640 to 960 x 960 pixels did not improve overall defect detection, with mAP50 decreasing from 0.681 to 0.650. A further increase to 1280 x 1280 pixels resulted in a larger reduction to 0.546. Inference time substantially increased, from 7.9 ms at 640 pixels, to 19.6 ms at 960 pixels and 44.8 ms at 1280 pixels. The results indicate the increasing resolution does not correspond to an improvement in detection accuracy. Despite the potential to retain greater detail, the higher resolution configurations performed worse at substantially greater inference times.

6.3 Selection of the Final Configuration

Based on the experimental results, YOLOv8s at 640 x 640 pixels was selected for the final model configuration. It achieved the highest overall validation performance across all four configurations investigated, while maintaining substantially lower inference times than the higher resolution configurations. Although greater recall was observed at 960 x 960 pixels, this came at the expense of mean average precision and greater inference times. The 640 x 640 pixel configuration as such provided the most preferable balance between computational cost and detection performance.

6.4 Practical Limitations and Engineering Suitability

The selected YOLOv8s configuration demonstrates potential for assisting in visual inspection of packaged pork rashers, with an average validation inference time of 7.9 ms/image. However, its test set recall of 0.437 indicates a considerable portion of actual defects were not detected. This is a significant limitation for an inspection application, where defective products could pass inspection. The model was evaluated using a static image dataset, not a physical production line. Further testing with larger more representative datasets, alongside evaluation under realistic operating conditions is necessary before considering industrial deployment.

7. Limitations, Lessons Learned and Future Work

7.1 Dataset and Validation Limitations

The dataset presented several limitations affecting the reliability of the evaluation. The defect classes were substantially imbalanced, with packaging error and unsealed defects accounting for 94.6% of annotations. The relatively small validation and test subsets containing 120 images and 80 images respectively, further limited the amount of evidence available for evaluating model performance. Additionally, neither subset contained annotated wrinkle defects, preventing evaluation of this class. An audit also identified potential overlap between source image groups across dataset splits, including five test-set images associated with overlapping source groups. This may reduce the independence of the held-out test set evaluation. From

this, the results should be interpreted within the limitations of the available dataset rather than interpreted as overall performance in an industrial food processing setting.

7.2 Model Limitations

The model demonstrated inconsistent performance across defect classes. While unsealed defects were detected relatively reliably, twisted-meat detection was particularly poor, with none of the four test instances correctly identified. Packaging error defects especially demonstrated poor localisation accuracy, with an mAP50-95 of 0.132. This highlights the model's difficulty detecting certain defect types reliably, despite its (comparatively) strong overall validation performance.

7.3 Lessons Learned

The experiments demonstrated that increasing model complexity or input resolution does not necessarily improve detection performance. The YOLOv8s at 640 x 640 pixels provided the strongest overall results, demonstrating the importance of balancing detection accuracy and computational cost. This project also highlighted how averaged performance metrics can conceal weaknesses in individual classes, reinforcing the importance of evaluating models at class-specific levels.

7.4 Future Improvements

Future work should prioritise expanding the dataset used and increasing the representation of underrepresented defect classes. Additional test data would also provide a more reliable assessment of generalisability. Further investigation into dataset augmentation, alternative model configurations and training strategies could improve detection performance across defect classes. Lastly, evaluation using images captured in realistic production-line conditions would be necessary to assess practical suitability.

8. Conclusion

This project investigated the application of YOLOv8 object detection to automated visual inspection of packaged pork rashers. Four model configurations were investigated across different model sizes and input resolutions. YOLOv8s at 640 x 640 pixels achieved the strongest overall validation performance, demonstrating the most favourable balance between detection accuracy and computational cost. However, evaluation on the test set revealed limitations in defect detection, particularly for certain defect classes. Overall, the investigation demonstrated the potential of computer vision for assisting packaging inspection while highlighting the importance of representative datasets, reliable validation and further development before practical application.

9. AI Use and Critical Reflection

ChatGPT was used throughout this project to assist with programming, debugging, researching technical concepts and structuring project documentation. AI assistance made a substantial contribution to the development process, particularly in explaining unfamiliar concepts and improving development efficiency. However, project direction and all engineering decisions remained my responsibility.

AI-generated code and technical explanations were checked through execution, examination of experimental results and comparison with external references and project documentation. Some suggestions required correction or were unsuitable for the project, reinforcing the importance of verifying AI-generated material.

I remained fully responsible for dataset evaluation, conducting experiments, interpreting results, model selection throughout and drawing engineering conclusions. Overall AI improved development efficiency and supported my understanding of machine learning concepts, while requiring critical and thorough evaluation of all material generated.

References

[1] - Roboflow, “YOLO annotation format”, *Roboflow.com,* 2025. https://roboflow.com/formats/yolo.
[2] - Ultralytics, “YOLO Architecture Explained”, *Ultralytics Docs,* July 07, 2026. https://docs.ultralytics.com/guides/yolo-architecture.
[3] - Ultralytics, “YOLO Performance Metrics”, *Ultralytics Docs, 2023.* https://docs.ultralytics.com/guides/yolo-performance-metrics
[4] - Roboflow, “Pork Rasher Error (Packaging) – Computer Vision Dataset,” Version 4, Roboflow Universe. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging/dataset/4.